CONTEXT-AWARE TRUST AND SENSOR DECISION SUPPORT FOR IoT SYSTEMS

PROJECT REPORT

Submitted by: Vishwas Kumar(23BCT0137) , Aakriti Tiwari(23BCT0168)

Course: Programming for IoT Boards

Project Guide: Dr. Pushpa Gothwal

Academic Year: 2023-2027

CONTENTS:

Sl. No.	Contents	Page No.
	Abstract	
1.	Introduction	2-3
1.1	Background	2-3
1.2	Motivation	3-4
1.3	Scope of the Project	4
2.	Project Description and Goals	4-5
2.1	Literature Review	5-6
2.2	Research Gap	6
2.3	Objectives	6
2.4	Problem Statement	7
2.5	Project Plan	7
3.	Technical Specification	7-10
3.1	Requirements	7
3.1.1	Functional Requirements	7-8
3.1.2	Non-Functional Requirements	8
3.2	Feasibility Study	8-10
3.2.1	Technical Feasibility	9
3.2.2	Economic Feasibility	9
3.2.3	Social Feasibility	9
3.3	System Specification	10
3.3.1	Hardware Specification	10
3.3.2	Software Specification	10
4.	Design Approach and Details	10-17
4.1	System Architecture	12
4.2	Design	14-17
4.2.1	Data Flow Diagram	14
4.2.2	Use Case Diagram	15
4.2.3	Class Diagram	16
4.2.4	Sequence Diagram	17
5.	Methodology and Testing	17-25

5.1	Module Description	17
5.2	Testing	25
6.	Project Demonstration	26
7.	Result and Discussion (Cost Analysis)	27-28
8.	Conclusion	28
9.	References	28-29

ABSTRACT

The Internet of Things (IoT) allows sensors and connected devices to collect information about the physical world. However, sensor readings are not always accurate or reliable. Different sensors may report different values because of changes in their surroundings, communication problems, old readings, or sensor faults. If a system directly follows a fixed threshold or calculates a simple average, it may make an incorrect decision.

This project presents a software-based prototype that checks the quality and reliability of sensor information before providing a decision recommendation. The system contains two separate workflows.

The first workflow is an industrial temperature-monitoring demonstration called PRESS-04. It uses four virtual temperature sensors to generate readings and related information such as timestamps, historical reliability, network quality, and operating context. The system validates the readings, detects conflicts, calculates the trustworthiness of each observation, estimates decision confidence, and selects one of three responses: execute an action, request additional data, or defer the decision for review. The simulated machine intent can be normal operation, warning, shutdown, or hold. This workflow demonstrates the decision logic only; it does not control real machinery.

The second workflow allows users to configure their own sensors by providing details such as sensor ID, measurement name, unit, location, and expected reporting interval. Readings can be entered through the dashboard or sent through a REST API. The system uses the first five readings to create an initial baseline for each sensor. Later readings are compared with recent history using the median and median absolute deviation (MAD). Based on this comparison, the system can identify readings as learning, usual, unusual, stale, or low quality. Each configured sensor is analyzed independently; readings from different measurement types are not combined into a single machine-level decision.

The application is developed using Python, FastAPI, Pydantic, SQLAlchemy, and SQLite for the backend, along with React, TypeScript, Vite, Tailwind CSS, and Recharts for the dashboard. Automated tests cover decision scenarios, confidence bands, sensor baseline behavior, stale data, invalid submissions, and database updates.

The current implementation is a local, single-workspace prototype. It does not include customer authentication, tenant separation, physical hardware connectivity, or safety certification. Therefore, its results should be considered software-based decision support for demonstration and evaluation, rather than a safety-critical control system.

Keywords: Internet of Things, Sensor Trust, Conflicting Readings, Data Freshness, Anomaly Detection, Sensor Baseline, Decision Confidence, FastAPI, React.

1. INTRODUCTION

1.1 Background

The Internet of Things (IoT) connects physical devices, sensors, communication networks, and software applications. Sensors measure physical conditions such as temperature, humidity, vibration, speed, and electrical current. These measurements are sent to software systems, where they can be stored, analyzed, and used to support monitoring and operational decisions.

However, a sensor reading cannot always be treated as the correct value. A sensor may become damaged, lose calibration, experience communication delays, or be affected by its installation location. As a result, two or more sensors measuring the same physical condition may produce different readings.

For example, consider an industrial machine monitored by three temperature sensors:

- Sensor A reports 75°C.
- Sensor B reports 73°C.
- Sensor C reports 30°C.

A basic monitoring system might shut down the machine whenever a temperature exceeds 70°C. Another system might calculate the average of all three readings, which is approximately 59.3°C.

Both methods can be misleading. The shutdown decision may be triggered by a faulty sensor, while the average may hide a real high-temperature condition. A better approach is to examine the quality of each reading before making a recommendation.

The system should consider questions such as:

- Is the reading recent?
- Has the sensor been reliable in the past?
- Does the reading match the sensor's recent behavior?
- Do other sensors report similar values?
- Is the network connection reliable?
- Is the sensor relevant to the current operating condition?

This project demonstrates two approaches to analyzing sensor information:

1. Multi-source temperature trust pipeline: Several simulated temperature sensors are compared, and a recommendation is made based on trust and confidence.
2. User-configured sensor baseline pipeline: A user defines a sensor, submits readings, and the system learns that sensor's normal range before identifying unusual or stale values.

These two workflows are designed for different purposes. The configurable sensor workflow supports different measurement names and units, but it currently checks each sensor against its own history. It does not combine different sensors into one general decision model.

1.2 Motivation

The main motivation for this project is to make IoT decision-making more reliable when sensor information is uncertain or conflicting.

A fixed threshold may not consider whether a reading is old or whether the sensor has a poor connection. Similarly, a simple average gives every sensor the same influence, even when some sensors are more reliable than others.

A single faulty reading may also differ significantly from the values reported by other sensors. In some cases, sensors may be divided into two groups, making a simple majority vote insufficient to resolve the disagreement.

The project is motivated by the following points:

- A fixed threshold does not consider sensor age or communication quality.
- A simple average gives unreliable sensors the same influence as reliable sensors.
- A faulty reading may strongly disagree with otherwise consistent sources.
- A split between sensor groups cannot always be resolved safely through majority voting alone.

- Users need a clear dashboard that explains the result instead of displaying only raw values or complex scores.
- A reusable software prototype should allow users to configure sensors and submit readings without requiring physical hardware during development.

The goal is to make the system's reasoning understandable. It should display the sensor readings, show the quality of the available evidence, and explain whether it recommends taking action, collecting more information, or asking for human review.

1.3 Scope of the Project

The project focuses on building a software prototype that demonstrates sensor trust, conflict detection, and decision support without requiring physical IoT devices.

Included in the current prototype

The current system includes:

- A simulated industrial machine and four virtual temperature sensors.
- Six simulation scenarios:
    - Normal operation
    - One faulty sensor
    - Split conflict
    - Poor network
    - Stale reading
    - Emergency
- Temperature preprocessing and validation.
- Freshness and communication-quality calculations.
- Temporal consistency and source agreement calculations.
- Contextual relevance evaluation.
- Conflict detection and sensor clustering.
- Trust-mass selection and confidence calculation.
- A decision controller with three responses: execute, request additional data, and defer.
- A React dashboard with separate pages for overview, sensor readings, scenarios, trends, sensor setup, and configured readings.
- REST API endpoints for simulator state, scenarios, sensor configuration, and configured-reading submission.
- SQLite storage for configured sensors, readings, simulator observations, and decisions.
- A per-sensor baseline demonstration that supports different measurement labels and units.
- Automated backend tests for important system behaviors.

Not included in the current prototype

The following features are outside the current implementation:

- Physical sensors, ESP32 firmware, gateway software, MQTT, or a device-to-cloud connector.
- Combining different user-configured sensors into a single decision.
- Automatic unit conversion.
- Validation that a selected unit matches the physical measurement.
- User accounts, roles, organization management, or separation of customer data.
- Production deployment, security assessment, load testing, or safety certification.
- Automatic control of real equipment.

These features may be considered in future development.

2. PROJECT DESCRIPTION AND GOALS

The project is developed as a local web application with a Python backend and a React-based dashboard. It demonstrates how an IoT system can evaluate sensor readings before making a recommendation.

The system has two main workflows:

- The PRESS-04 workflow uses simulated temperature sensors to demonstrate trust-aware decision-making when multiple sources disagree.
- The configurable sensor workflow allows users to create their own sensor profiles, submit readings, and check those readings against a learned baseline.

The main aim is not to permanently label a sensor as good or bad. In the PRESS-04 workflow, trust is recalculated for each observation using its history and current conditions. In the configurable sensor workflow, each reading is compared with the sensor's recent history and communication quality.

2.1 Literature Review

IoT Systems and Connected Devices

Atzori, Iera, and Morabito describe the Internet of Things as an ecosystem of connected objects, technologies, and applications [1]. Their work provides the background for understanding how sensors, communication systems, and software services work together in an IoT application.

The present project follows this general idea by representing the sensor and application layers through software. However, the current prototype does not connect to physical devices.

NIST Special Publication 800-183 explains the concept of a "Network of Things" using system components such as sensing, computing, communication, and actuation [2]. It also discusses the challenges created by different devices and the need to consider reliability and security.

This supports the idea that a sensor reading should be understood in the context in which it was generated, rather than being treated as an isolated number. In the PRESS-04 workflow, context includes information such as the operating mode, sensor location, signal quality, and timestamp.

Trust and Reputation in Sensor Networks

Khalid et al. present a survey of trust and reputation systems for wireless sensor networks [3]. This area of research examines how a system can evaluate the behavior of connected nodes and the reliability of information when devices or network connections may not always work correctly.

The project uses this general concept in its temperature-monitoring workflow. Instead of relying only on a sensor's reported value, it considers historical reliability along with current temporal behavior, agreement with other sensors, freshness, communication quality, and context.

Robust Statistical Methods for Unusual Readings

The configurable sensor workflow uses the median and median absolute deviation (MAD) to estimate a sensor's normal behavior.

These statistical methods are useful when a dataset may contain extreme values. The median is less affected by unusually high or low observations than the arithmetic mean.

Rousseeuw and Croux discuss robust alternatives to the median absolute deviation and the use of robust scale estimators [4]. The current implementation uses a MAD-based calculation as a practical choice for the prototype. It is not a complete statistical calibration study.

Technologies Used in the Web Application

The project uses FastAPI to expose Python functions through HTTP endpoints and provide API documentation [5]. React is used to build the dashboard with reusable components [6].

SQLAlchemy maps application objects to database tables [7], while SQLite provides a local, file-based database for storing the prototype's data [8].

Summary of the Literature Review

The reviewed work supports three important ideas:

1. IoT systems contain different devices, sensors, and communication networks.
2. Trust can help evaluate the changing reliability of devices and sensor information.
3. Robust statistical methods can reduce the influence of extreme readings.

The project brings these ideas together in a software demonstration. However, the specific trust formula and thresholds used in the prototype are engineering choices. They are not claimed to be scientifically validated or proven to be novel.

2.2 Research Gap

The main problem addressed by this project is the gap between receiving sensor readings and making a cautious, understandable recommendation when the available information is uncertain or conflicting.

Many basic monitoring systems use a fixed threshold or calculate an average of the available readings. For the scenarios considered in this project, these methods do not clearly show whether a reading is stale, has poor communication quality, differs from its own recent history, or belongs to a competing group of sensors.

To address this issue, the project demonstrates a processing pipeline that evaluates trust, conflict, and confidence before recommending a response.

The configurable sensor workflow addresses a related but narrower problem. It allows users to store their own sensor information and check readings against a learned baseline without requiring a measurement-specific threshold to be built into the frontend.

However, this workflow is not yet a general multi-sensor fusion system. Each configured sensor is evaluated independently. The system does not combine temperature, vibration, humidity, or other measurements into one machine-level risk score.

This research gap represents the design motivation for the project. It does not mean that no existing research or product has already addressed these problems.

2.3 Objectives

The main objectives of the project are:

1. Develop a software-based IoT simulation that can run without physical sensors.
2. Generate multiple virtual temperature readings with timestamps, historical reliability, communication quality, and operating context.
3. Detect disagreements between temperature sensors and group similar readings.
4. Calculate sensor trust dynamically using historical reliability and current observation quality.
5. Generate a confidence score and select one of three responses: EXECUTE, REQUEST_DATA, or DEFER.
6. Allow users to configure sensors using their own measurement names and units.
7. Accept sensor readings through the dashboard or a REST API.
8. Learn a separate baseline for each configured sensor and identify unusual, stale, or low-quality readings.
9. Display sensor information and recommendations through a clear, user-friendly dashboard.

10. Verify the system's important behaviors using automated backend tests and a frontend production build.

2.4 Problem Statement

When several sensors measure the same physical quantity, their readings may differ because of sensor faults, communication problems, environmental conditions, or delays.

The system must determine whether the readings are usable, whether they disagree, and which group of readings has stronger supporting evidence. Based on this information, it must provide a recommendation that reflects the quality and confidence of the evidence.

The system should not blindly follow a single sensor or calculate a simple average without considering the reliability of the sources.

Separately, when a user configures a sensor with a numeric measurement and unit, the system must store its readings, learn its normal range from initial data, and identify whether a new reading is usual, unusual, stale, or received through a weak connection.

The two workflows must remain clearly separated. The configurable sensor baseline system is not the same algorithm as the PRESS-04 multi-source trust engine.

2.5 Project Plan

The project was divided into the following development phases:

Phase	Planned Work	Status
1	Define the IoT decision problem and software scope	Completed
2	Create virtual sensor profiles and fault scenarios	Completed
3	Validate and preprocess temperature readings	Completed
4	Detect conflicts and group similar readings	Completed
5	Calculate context-aware trust and confidence	Completed
6	Select execute, request-data, or defer response	Completed
7	Add FastAPI endpoints and SQLite storage	Completed
8	Develop dashboard pages and charts	Completed
9	Add user-configured sensor information and REST ingestion	Completed
10	Add per-sensor baseline checks and stale-data status	Completed
11	Test backend behavior and build the frontend	Completed
12	Add physical device connectivity, user accounts, tenant separation, and deployment improvements	Future Work

3. TECHNICAL SPECIFICATION

3.1 Requirements

The requirements describe the functions the system must provide and the quality standards it should follow.

3.1.1 Functional Requirements

ID	Functional Requirement
FR-01	The system must generate virtual sensor readings for the PRESS-04 demonstration.

FR-02	The system must provide named simulation scenarios that introduce faults or conflicting readings.
FR-03	The system must validate and preprocess temperature readings, including their age and communication information.
FR-04	The temperature engine must compare valid readings, detect conflicts, and group readings based on temperature similarity.
FR-05	The temperature engine must calculate trust scores using historical and current evidence.
FR-06	The controller must choose whether to execute, request more data, or defer based on decision confidence.
FR-07	The REST API must provide simulator state, history, scenario selection, and raw temperature-reading submission.
FR-08	Users must be able to add a sensor using its ID, name, measurement, unit, location, and reporting interval.
FR-09	The API must reject duplicate configured sensor IDs and readings submitted for unknown sensor IDs.
FR-10	Users or external clients must be able to submit a configured sensor reading with an optional network-quality value and timestamp.
FR-11	The configured sensor workflow must learn a baseline, identify unusual changes, and detect stale or low-quality readings.
FR-12	The dashboard must display the PRESS-04 demonstration and configurable sensor workflow on separate pages.
FR-13	The system must store configured sensor information and readings in SQLite.
FR-14	When a configured sensor is removed, its associated saved readings must also be removed.

3.1.2 Non-Functional Requirements

ID	Non-Functional Requirement
NFR-01	The application must run on a developer’s computer without physical IoT hardware.
NFR-02	The REST API must use JSON for request and response data.
NFR-03	The sensor setup form must check required fields before submitting information.
NFR-04	The backend must return clear HTTP errors for invalid or unknown sensor submissions.
NFR-05	The dashboard must use clear labels and organize tasks into navigable pages.
NFR-06	Database initialization must create missing tables and update the configured-sensor table when necessary.
NFR-07	Backend decision behavior must be covered by repeatable automated tests.
NFR-08	The results must be presented as prototype recommendations, not certified safety controls.
NFR-09	The local application must not be treated as a multi-customer hosted system because authentication and tenant separation are not implemented.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project can be developed using commonly available software development tools.

Python is used for the backend API and decision-making logic. FastAPI provides the REST endpoints, while Pydantic validates the incoming data. SQLAlchemy and SQLite are used for local data storage.

The frontend is developed using React and TypeScript. Recharts is used to display historical data through charts.

The main processing tasks—generating simulated sensor readings, comparing temperature values, calculating trust, detecting conflicts, and checking per-sensor baselines—are suitable for a local software demonstration.

The application has been checked using automated tests and a production frontend build.

However, these checks do not prove that the application is ready for production deployment. The system has not been evaluated for multiple concurrent customers, high-volume data ingestion, advanced security requirements, or real equipment safety.

Connecting physical devices and supporting separate customer workspaces would require additional development, architecture, and testing.

3.2.2 Economic Feasibility

The software demonstration does not require the purchase of physical sensors, an ESP32, a message broker, or a cloud account.

The project uses a local SQLite database and the software packages listed in the repository. This makes it inexpensive to run as an academic prototype on an existing computer.

The report does not provide a commercial hosting cost or total product development cost. These costs would depend on the hosting provider, data volume, storage requirements, number of devices, support requirements, security needs, and software licenses.

A commercial version would also require investment in authentication, customer data separation, monitoring, backups, operational support, and security review.

Cost Category	Current Prototype
Physical sensors	Not required
Local database	SQLite file; no separate database server
Backend and frontend runtime	Existing computer and installed software
Cloud hosting	Not included or priced
Commercial operations	Not included or priced

3.2.3 Social Feasibility

The project aims to help users understand when sensor readings may be unreliable and why the software recommends collecting more information or requesting human review.

Displaying a clear status and explanation can help users make better-informed decisions than simply showing raw sensor values.

However, users should not treat a calculated trust or confidence score as absolute certainty. Incorrect baselines, poor sensor installation, biased data, or misunderstood units can lead to incorrect results.

The interface should therefore retain human oversight, clearly identify simulated measurements, and avoid presenting the prototype as a certified safety system.

3.3 System Specification

3.3.1 Hardware Specification

Hardware Required to Run the Prototype

The software can run on a personal computer or laptop with the following:

- A supported Python interpreter.
- Node.js for the frontend.
- A modern web browser.
- Network access to localhost so the frontend can communicate with the backend.
- Sufficient disk space for the source code, dependencies, and SQLite database.

Physical IoT Hardware

No physical sensors, microcontroller, gateway, or actuator is required for the current prototype.

A future version may connect physical devices through REST or another communication method. However, that connector is outside the current implementation.

The repository does not define or measure minimum CPU, memory, or disk requirements. These requirements would need to be measured if the software is packaged for deployment.

3.3.2 Software Specification

Layer	Technology	Purpose
Backend language	Python	Simulator, validation, trust logic, and baseline checks
API framework	FastAPI	REST endpoints and API documentation
Data validation	Pydantic	Validates request and observation data
ORM	SQLAlchemy	Database models and queries
Database	SQLite	Stores readings, decisions, sensors, and history locally
Frontend	React + TypeScript	Dashboard pages and user interaction
Development and build tool	Vite	Frontend development server and production build
Styling	Tailwind CSS	Dashboard layout and styling
Charts	Recharts	Displays temperature and confidence history
Testing	pytest	Backend unit tests and behavior tests

Backend package version requirements are listed in backend/requirements.txt. Frontend dependencies and the lockfile are stored in frontend/package.json and frontend/package-lock.json.

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

The system contains two separate processing paths. Both paths use the API, database, and dashboard, but their sensor-analysis methods are different.

Path 1: PRESS-04 Temperature Demonstration

The first path uses virtual temperature sensors to demonstrate how the system handles conflicting readings.

The processing steps are:

1. Generate virtual sensor readings.
2. Validate and preprocess the observations.
3. Detect conflicts and group similar temperatures.
4. Calculate context-aware trust.
5. Calculate decision confidence.
6. Recommend whether to execute, request more data, or defer.

Path 2: User-Configured Sensor Workflow

The second path allows a user to configure a sensor and submit readings.

The processing steps are:

1. Create a sensor profile.
2. Receive a reading through the dashboard or REST API.
3. Validate the sensor ID, reading, timestamp, and connection quality.
4. Compare the reading with the sensor's own recent history.
5. Calculate the baseline and check for unusual behavior.
6. Display the result on the configured-sensor dashboard.

System Architecture Diagram

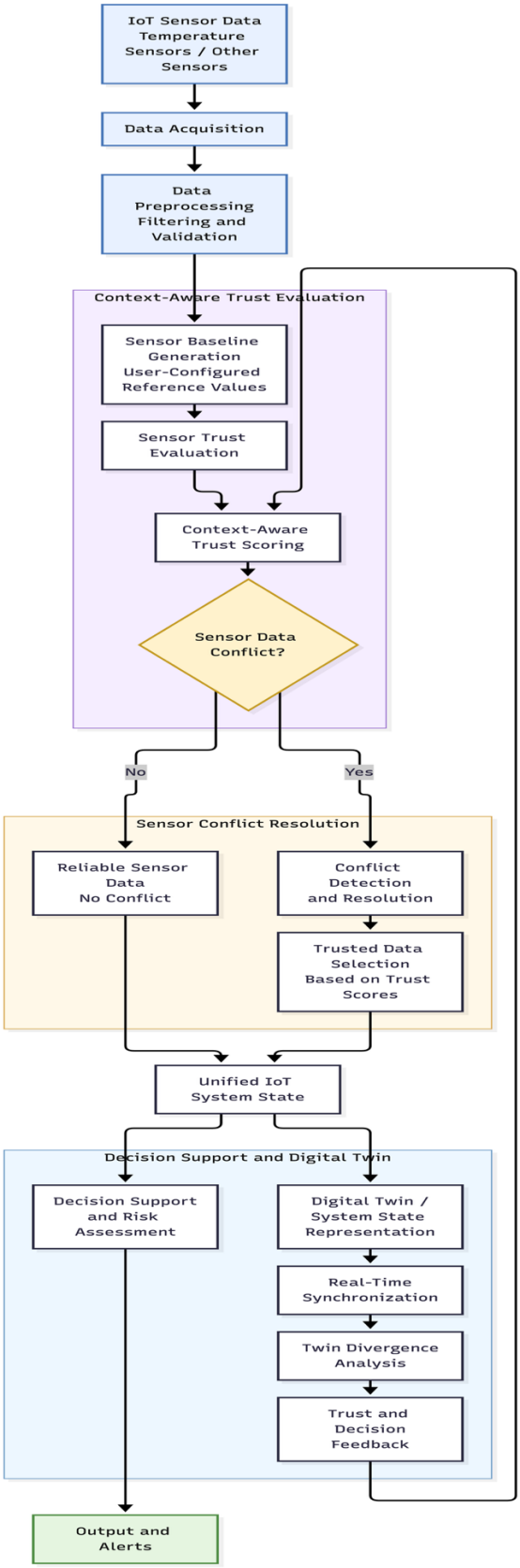


Figure 4.1 — Overall Architecture of the IoT Trust and Sensor Decision Support System

The simulator runs its own PRESS-04 cycle. Configured sensors store their own definitions and readings separately.

An unusual reading from a configured sensor is only a status indication. It is not a command to shut down equipment.

4.2 Design

The backend is divided into separate components for the simulator, temperature decision engine, configured-sensor service, API routes, database models, and database initialization.

The frontend also separates the PRESS-04 demonstration pages from the user-configured sensor pages. This makes it easier for users to understand the purpose of each workflow.

4.2.1 Data Flow Diagram

Level 0: Context Diagram

The Level 0 data flow diagram shows the system as a single application and identifies the external entities that interact with it.

The user can configure sensors, select demo scenarios, enter readings, and view recommendations. External devices or integrations can submit configured sensor readings through the REST API.

The application sends sensor statuses, recommendations, and history back to the user or external device. It also stores and retrieves information from the SQLite database.

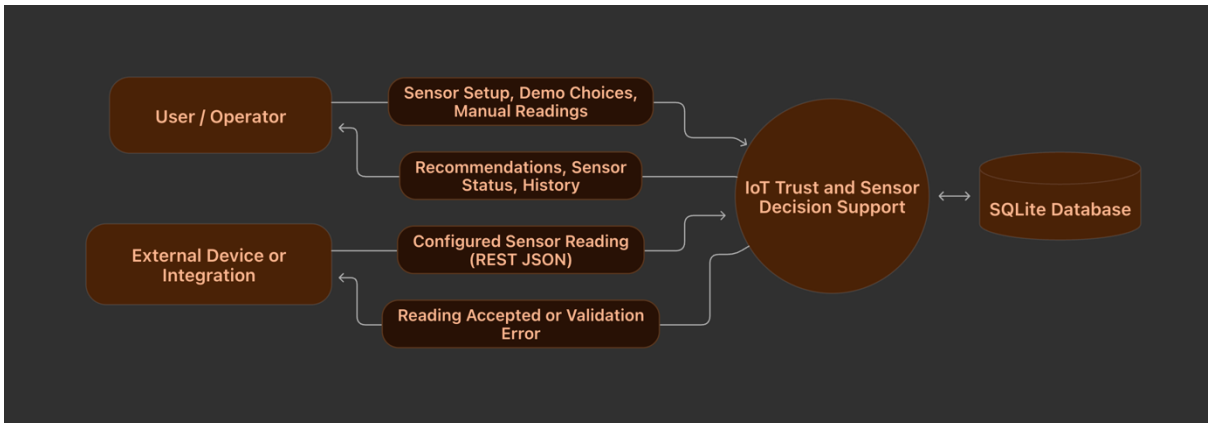


Figure 4.2 — Level 0 Data Flow Diagram

Level 1: Main Processing Flow

The Level 1 data flow diagram explains how the application processes the two types of input.

When a sensor is configured, its information is stored in the database. A reading can then be received through either the PRESS-04 demo or the configurable sensor workflow.

For the PRESS-04 demo, the system validates and preprocesses the observation, compares the temperature sources, calculates trust and confidence, and produces a recommendation.

For a configured sensor, the system checks the sensor ID, stores the timestamped reading, evaluates its age and network quality, and compares it with the sensor's own baseline. The result is then displayed on the dashboard.

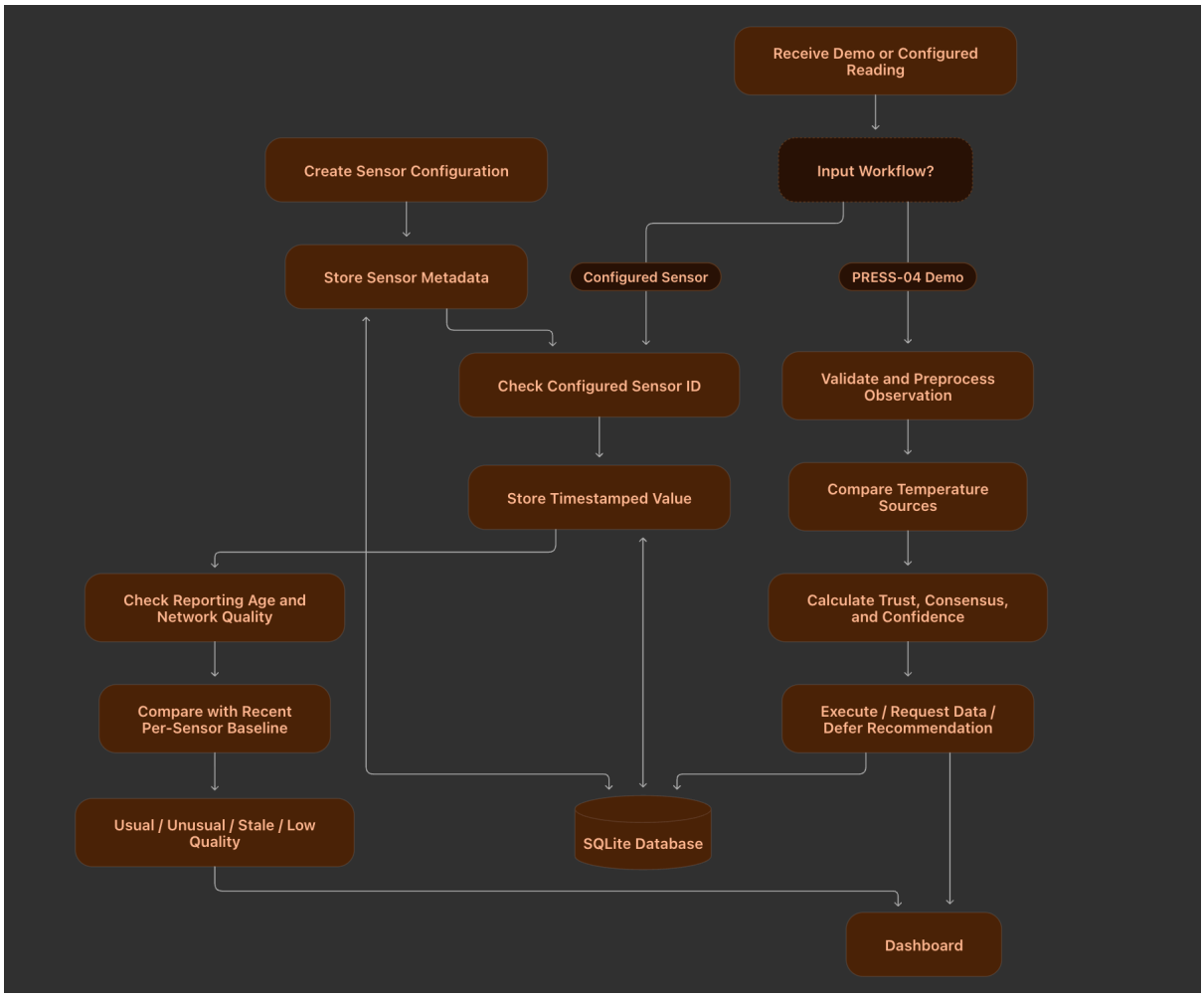


Figure 4.3 — Level 1 Data Flow Diagram

4.2.2 Use Case Diagram

The use case diagram shows the actions available to the user and to an external sensor device or integration.

The user can view the demo overview, inspect virtual sensor readings, select scenarios, view historical charts, configure sensors, remove configured sensors, enter readings, review baseline results, and check the service status.

An external device or integration can submit a configured sensor reading through the REST API.

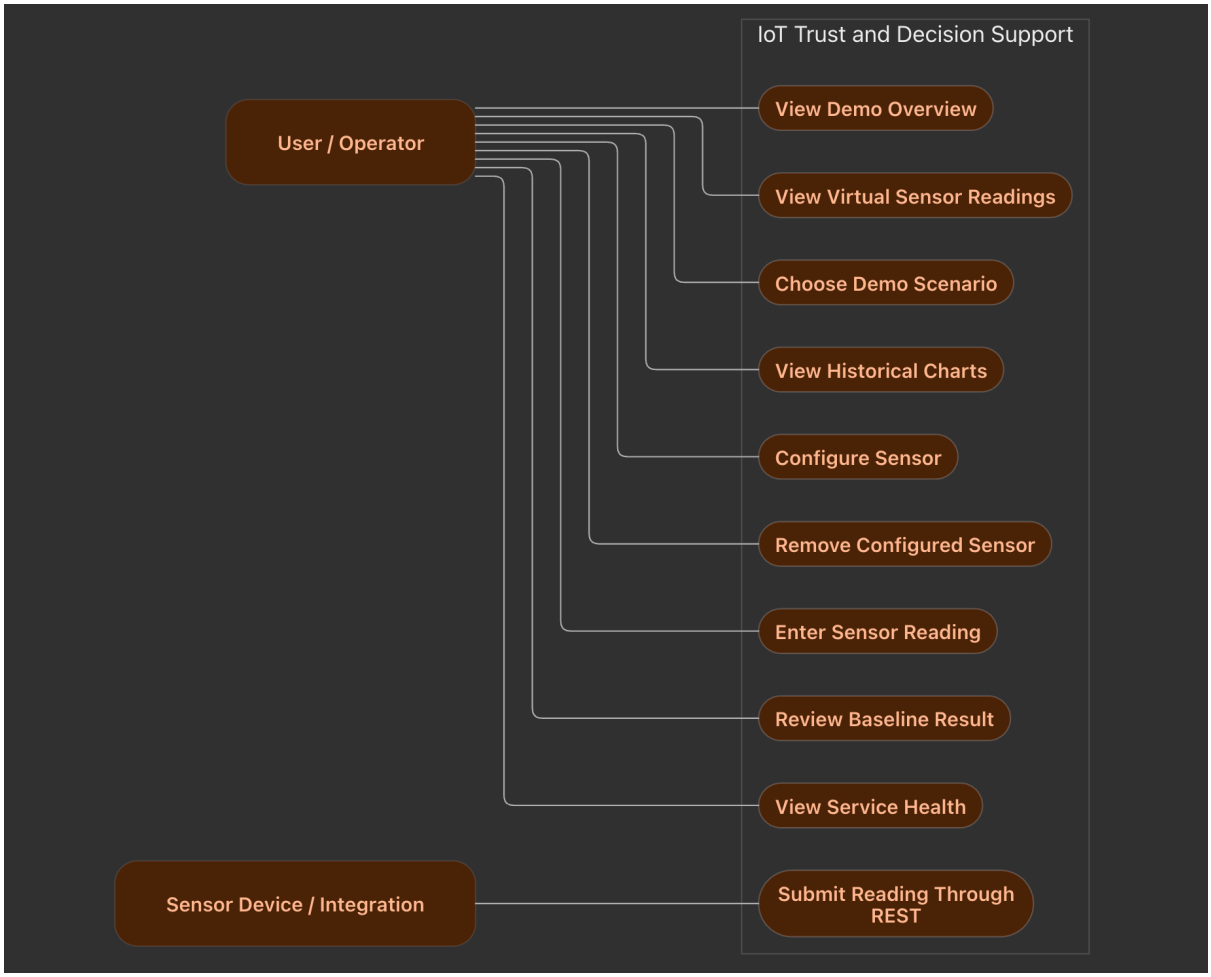


Figure 4.4 — Use Case Diagram

4.2.3 Class Diagram

The class diagram represents the main data structures and software components used in the application.

The Observation class stores the information generated by a virtual sensor. The IoTSimulator generates observations and applies different scenarios. The DecisionPipeline processes the observations and produces a DecisionOutcome.

The configured-sensor workflow uses separate database models for sensor information and sensor readings. Each configured sensor can have multiple saved readings.

The Runtime component controls the simulator and decision pipeline, while the FastAPI application exposes the system's functionality through REST endpoints.

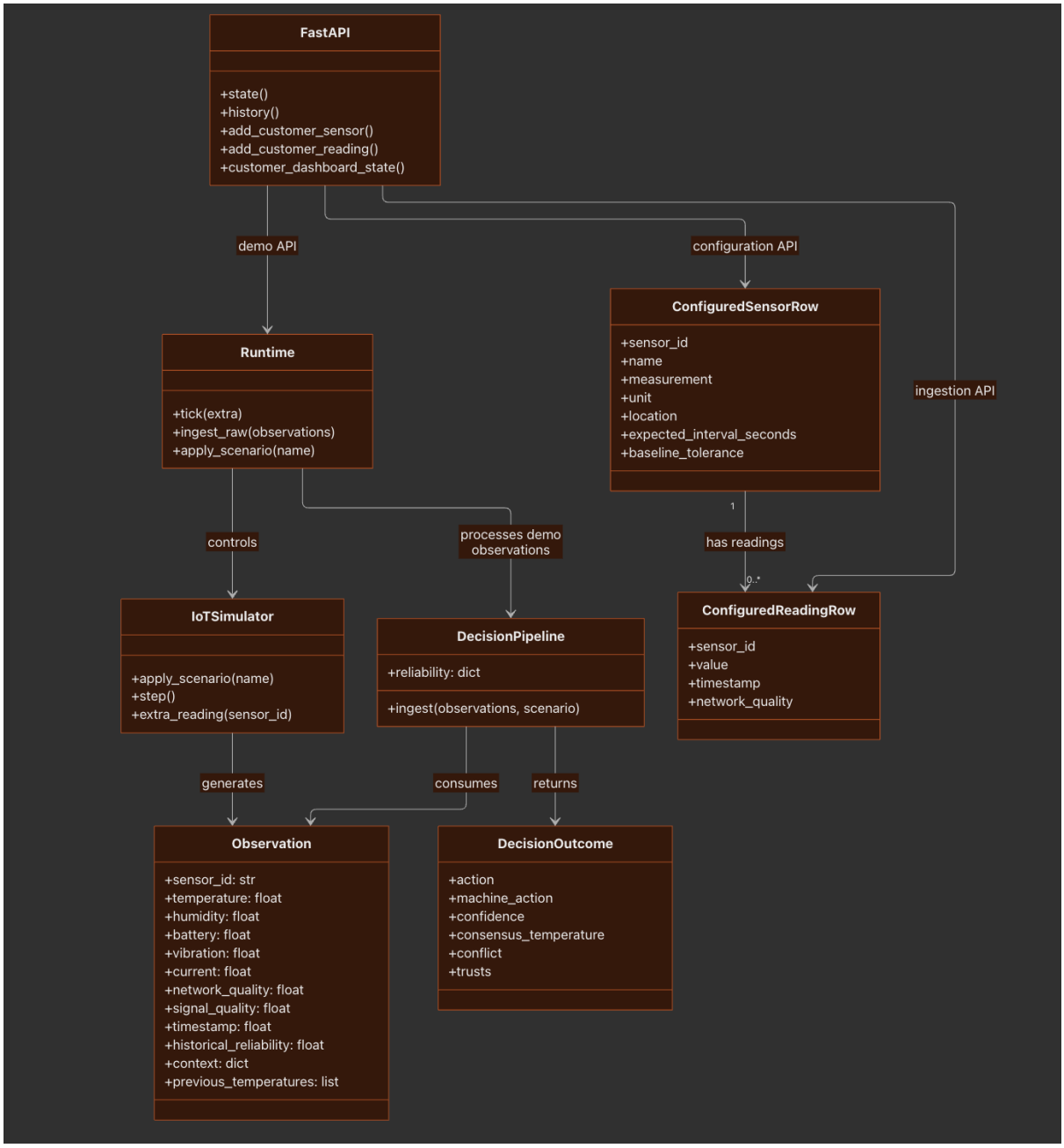


Figure 4.5 — Class Diagram

4.2.4 Sequence Diagram

The sequence diagram shows how a user configures a sensor and submits a reading.

First, the user enters the sensor details through the React dashboard. The dashboard sends a request to FastAPI, which validates the information and asks the configured-sensor service to create the sensor. The sensor information is then saved in SQLite.

Next, the user submits a numeric reading. The API validates the sensor ID and reading, and the service saves the timestamped value in the database.

When the dashboard requests the configured sensor state, the service reads the sensor definitions and recent readings, evaluates the baseline and data quality, and returns the results to the dashboard.

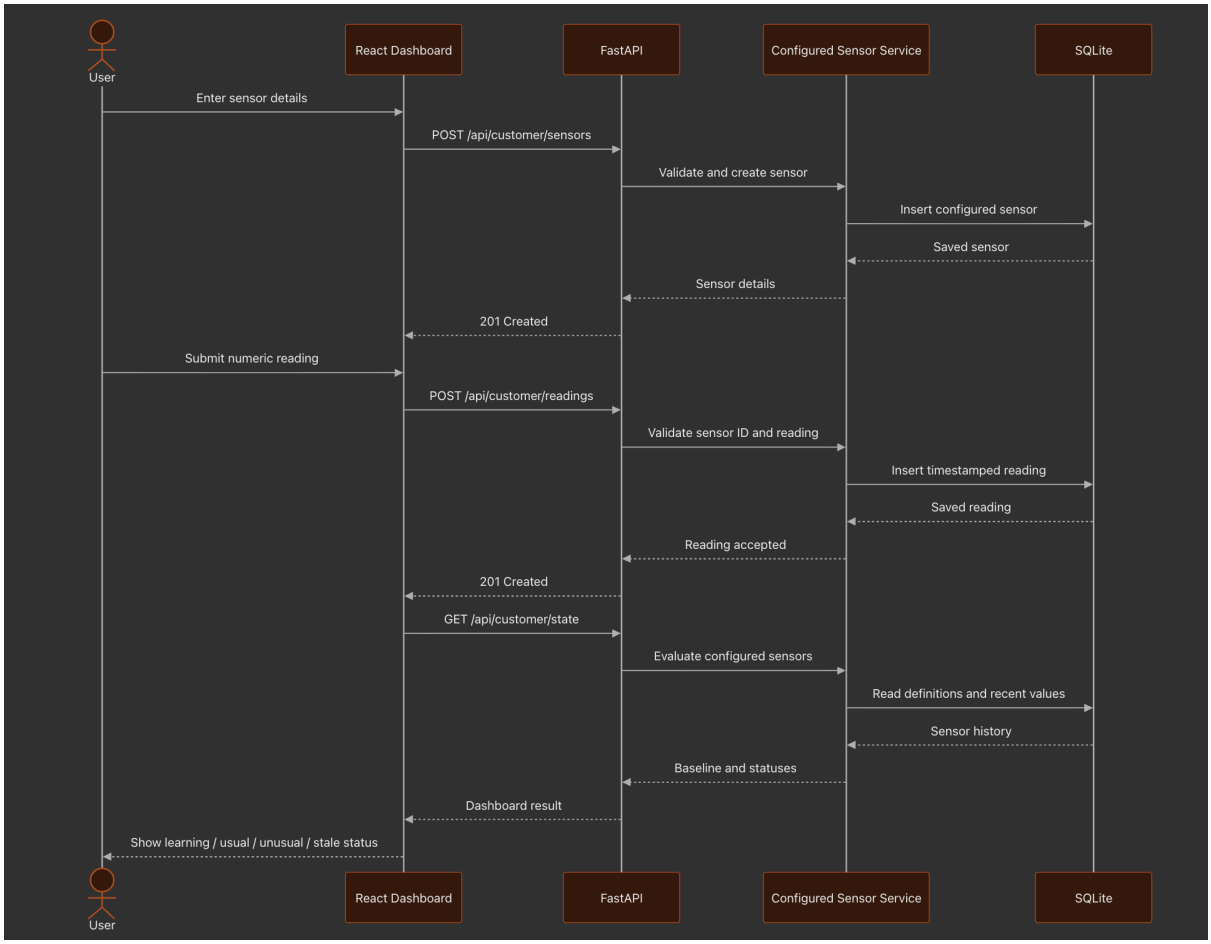


Figure 4.7 — Sequence Diagram

5. METHODOLOGY AND TESTING

5.1 Module Description

The system is divided into modules that handle sensor simulation, data preprocessing, conflict detection, trust calculation, confidence calculation, configured sensor analysis, API communication, database storage, and dashboard presentation.

5.1.1 Virtual Sensor Simulator

The simulator contains four virtual sensor profiles named S01, S02, S03, and S04.

Each sensor generates readings around a simulated machine temperature. Small random variations are added to make the readings more realistic.

The sensor profiles include information such as:

- Sensor location
- Sensor role
- Historical reliability
- Previous readings
- Measurement noise

The simulator can also introduce different conditions by selecting a scenario. These scenarios can create a sudden temperature spike, a stuck-low reading, an old timestamp, or poor network and signal quality.

The simulated true temperature is available for demonstration purposes. However, the trust engine does not use this true temperature to make its decision.

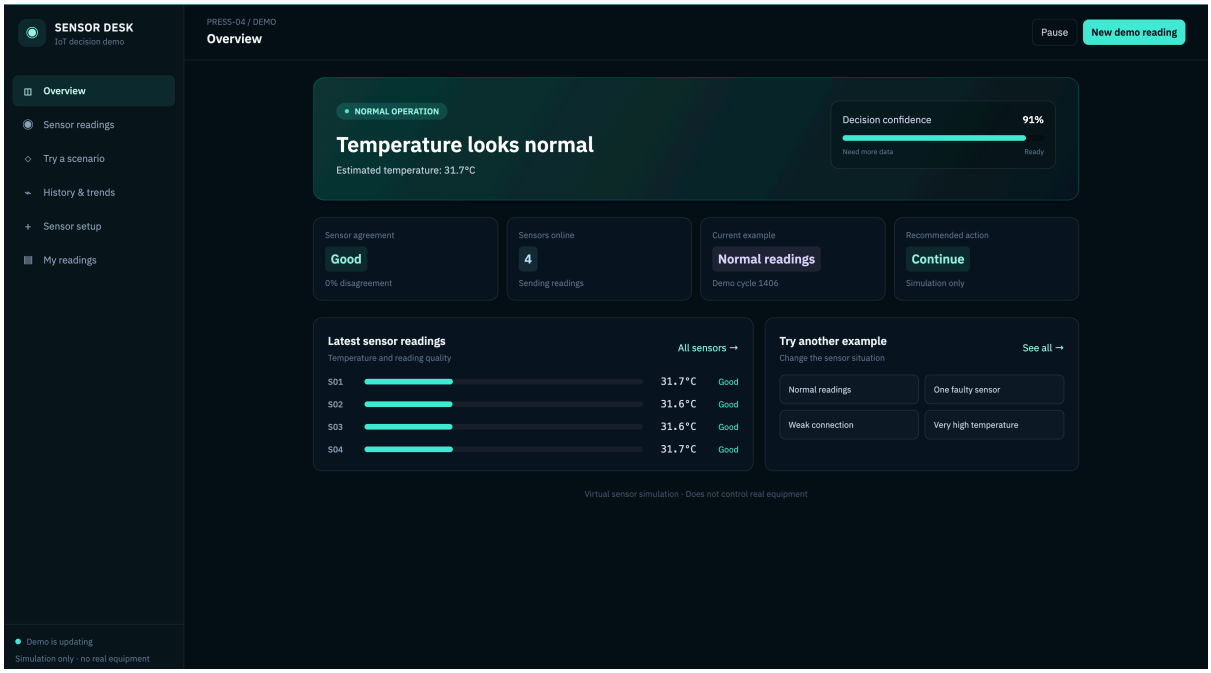


Figure 5.1 — Virtual Sensor Simulator

5.1.2 Temperature Preprocessing

Before the temperature readings are used by the decision engine, the system checks whether they are valid.

The temperature must be a finite number and must fall within the configured physical range of −20°C to 200°C.

The system also calculates how fresh each reading is. Freshness decreases as the reading becomes older.

The freshness calculation is:

$$\text{freshness} = \exp(-\max(0, \text{age_seconds}) / \text{freshness_tau_seconds})$$

The freshness time constant used in the prototype is 25 seconds.

Readings that are at least 40 seconds old are marked as stale in their notes.

Communication quality is calculated using both network quality and signal quality. The two input values are limited to the range from 0 to 1.

The calculation is:

$$\text{communication_quality} = \sqrt{\text{network_quality} \times \text{signal_quality}}$$

This helps the system account for both the network connection and the quality of the received signal.

5.1.3 Conflict Detection and Clustering

The temperature decision engine compares every pair of valid sensor observations.

A pair of readings is marked as conflicting when the difference between their temperatures is at least 8°C.

The system also calculates pairwise agreement using exponential decay with a 4°C scale. This allows the system to represent how similar two temperature readings are.

The readings are then sorted by temperature and grouped together when adjacent readings are less than the conflict gap apart.

Based on the fraction of conflicting pairs and the resulting cluster pattern, the system reports the conflict level as:

- Low
- Medium
- High

The purpose of this module is to identify whether the sensors are reporting a similar temperature or whether the readings form separate groups.

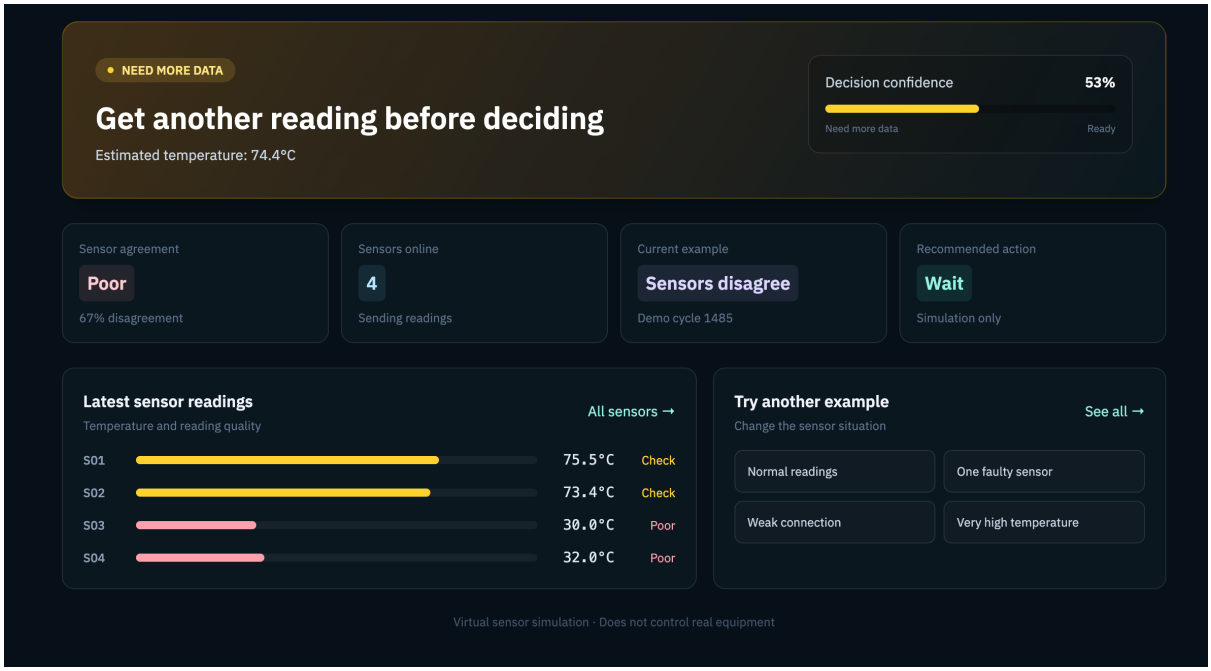


Figure 5.2 — Conflict Detection and Sensor Clustering

5.1.4 Context-Aware Trust Calculation

The trust engine evaluates each valid temperature observation using both historical information and current evidence.

The main factors considered are:

Temporal consistency: Checks whether the new reading is close to the sensor's previous readings and whether the sensor has made a sudden change.

Source agreement: Measures how closely the sensor's reading matches the other valid temperature observations.

Freshness: Evaluates how recently the reading was received.

Communication quality: Considers the network and signal quality associated with the observation.

Contextual relevance: Adjusts the evaluation based on the sensor's role, location, machine operating mode, battery state, and observation age.

Historical reliability: Represents the sensor's earlier reliability. This value is updated over time following sufficiently confident decisions.

The current implementation uses the following calculation pattern:

evidence_quality = freshness^1.15 × communication_quality^1.05
prior_weight = 0.55 × (1 − evidence_quality) + 0.22
live_evidence = 0.55 × temporal_consistency + 0.45 × source_agreement
posterior_score = prior_weight × historical_reliability + (1 − prior_weight) × live_evidence
trust_score = evidence_quality × posterior_score × (0.35 + 0.65 × contextual_relevance)

The trust score is bounded to its allowed range and classified as trusted, uncertain, or low trust.

Invalid observations receive a trust score of zero.

This formula is an engineering heuristic used in the current prototype. It is not a statistically calibrated probability, and the formula itself is not claimed to be a proven scientific result.

5.1.5 Confidence and Decision Controller

After calculating trust scores, the system groups the observations into temperature clusters.

The trust scores within each cluster are added together. This total is called the cluster's trust mass.

The cluster with the highest trust mass is selected as the dominant group. The system then calculates a trust-weighted temperature for that group.

Decision confidence is calculated using several factors:

- The trust mass of the dominant group.
- The level of agreement within that group.
- The quality of the available evidence.
- How clearly the temperature supports a particular machine intent.
- A penalty when another competing cluster also has substantial trust.

The controller uses the following confidence ranges:

Confidence	System Response
At least 0.80	Execute the machine intent in the simulation
From 0.50 to below 0.80	Request more sensor data and hold the machine intent
Below 0.50	Defer the decision and request additional information or human review

The machine intent is determined from the trust-weighted temperature consensus:

Temperature	Simulated Machine Intent
Below 70°C	Normal
At least 70°C	Warning
At least 78°C	Shutdown

These temperature thresholds are demonstration settings. They are not validated industrial operating limits.

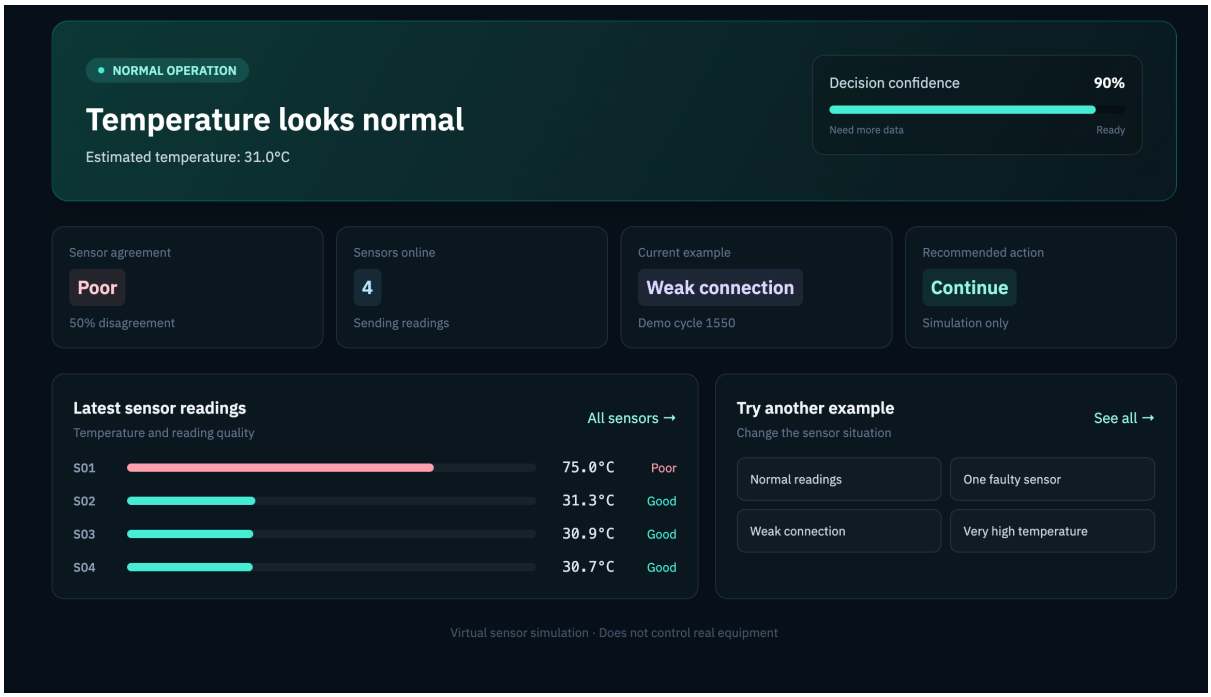


Figure 5.3 — Decision Confidence and Response Selection

5.1.6 Configured Sensor and Baseline Service

The configurable sensor workflow allows users to create a sensor profile using:

- Unique sensor ID
- Sensor name
- Measurement name
- Unit
- Location
- Expected reporting interval
- Optional baseline tolerance

The sensor information and its timestamped readings are stored separately from the PRESS-04 demonstration data.

The first five readings are used to create an initial baseline.

Beginning with the sixth reading, the system compares the latest value with previous readings from the same sensor. The comparison uses a history window of up to 30 readings.

The baseline centre is calculated using the median.

The robust spread and deviation score are calculated as follows:

$$\text{spread} = 1.4826 \times \text{median}(\text{abs}(\text{previous_value} - \text{baseline_median}))$$

$$\text{deviation_score} = \text{abs}(\text{latest_value} - \text{baseline_median}) / \text{spread}$$

If the calculated spread is effectively zero, the system uses the optional sensor tolerance. Otherwise, it applies a small fallback spread based on the median.

A deviation score of at least 3.5 is marked as unusual.

The dashboard can also show the following states:

- Learning
- No data
- Stale data
- Low connection quality

Freshness is evaluated using the expected reporting interval. A reading older than three expected reporting intervals is marked as stale.

A network-quality value below 0.3 is considered weak.

These thresholds are initial prototype settings. They must be checked and adjusted according to the actual sensor, measurement unit, and use case before any operational use.

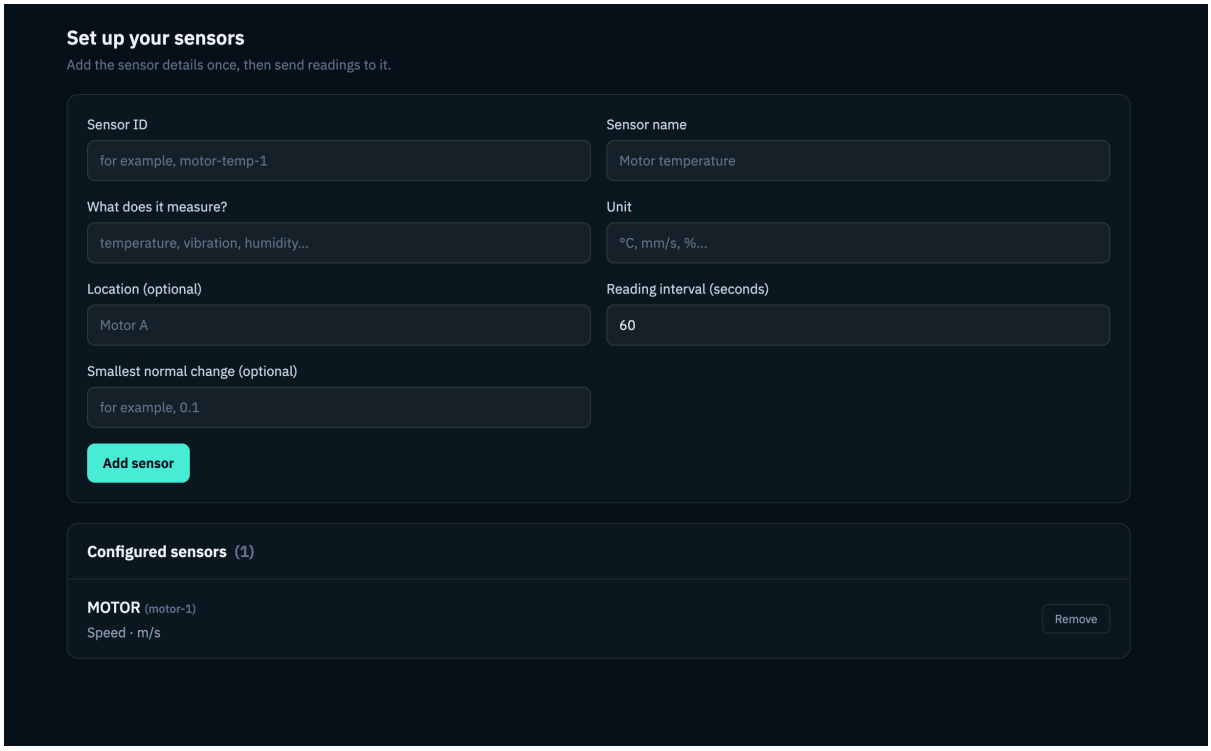


Figure 5.4 — Configured Sensor Setup

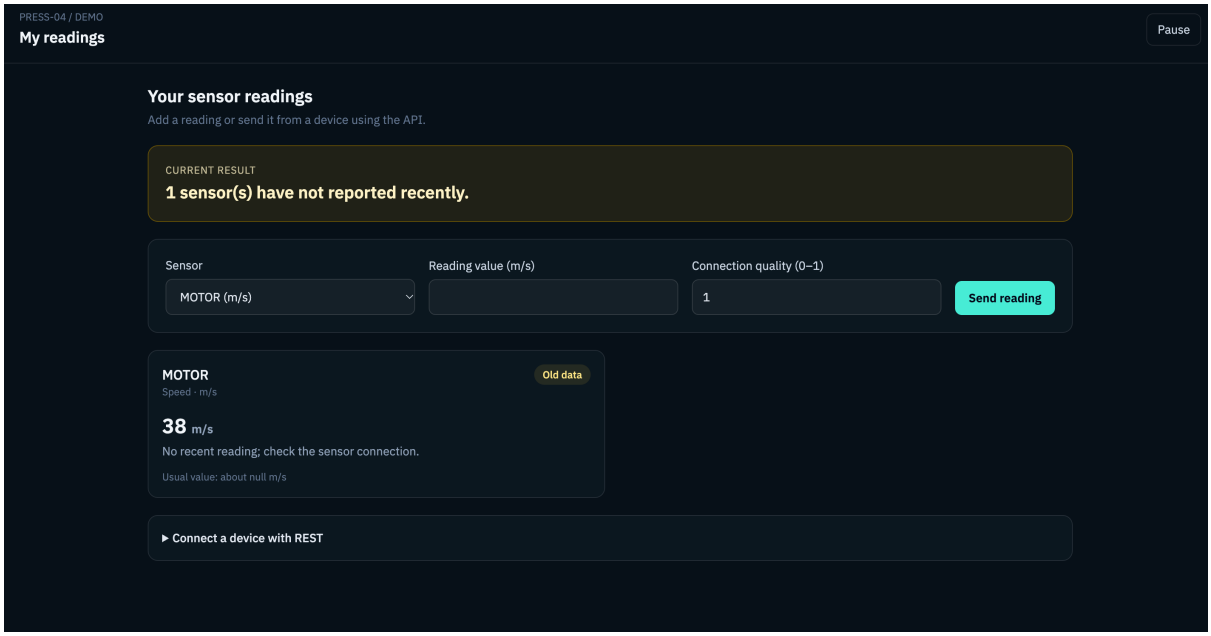


Figure 5.5 — Sensor Baseline and Reading Evaluation

5.1.7 API and Database Persistence

FastAPI provides REST endpoints for:

- Simulator state
- Scenario selection
- Historical readings
- Sensor configuration
- Configured sensor readings
- Configured sensor results

Pydantic schemas validate the incoming data and check required fields and input ranges.

SQLAlchemy models represent the readings, decisions, configured sensors, and configured sensor readings. These records are stored in the SQLite database.

The database initialization process creates the required tables. It also applies a small database migration to add the reporting-interval column to older configured-sensor databases when needed.

5.1.8 Dashboard

The React dashboard organizes the application into separate pages:

- Overview: Displays the main system information.
- Sensor Readings: Shows the virtual sensor readings.
- Try a Scenario: Allows the user to select a simulation scenario.
- History & Trends: Displays changes over time through charts.
- Sensor Setup: Allows the user to configure a sensor.
- My Readings: Displays readings and baseline results for configured sensors.

The dashboard presents recommendations and simplified sensor statuses in an understandable way. More detailed algorithm information is kept separate so that users can focus on the result.

The dashboard is a demonstration interface. It is not intended to function as an alarm panel for real industrial machinery.

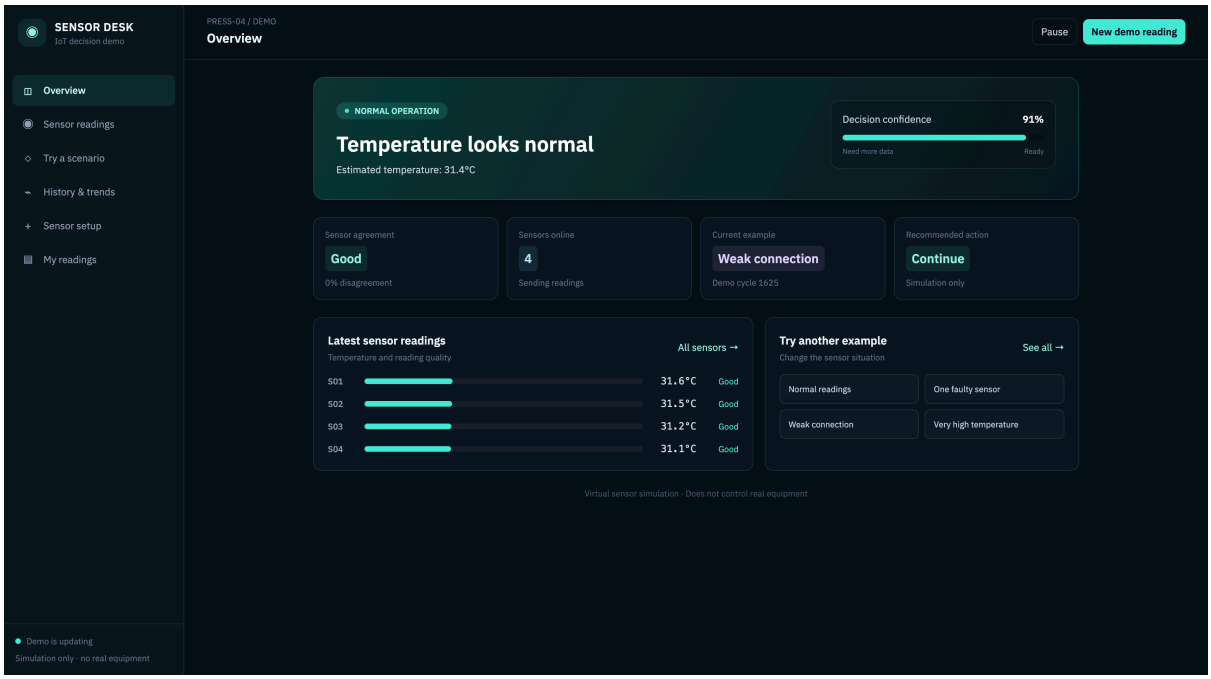


Figure 5.6 — Main Dashboard Overview



Figure 5.7 — Sensor History and Trend Visualization

5.2 Testing

Testing was carried out to check whether the system behaves as expected under different sensor conditions.

Test Approach

The backend tests use controlled sensor observations, a temporary SQLite database, and assertions on the system's outputs.

The tests check both the expected status and important decision properties.

The frontend is checked using its TypeScript production build and Oxlint.

Tested Behaviours

Test Group	Behaviour Verified
PRESS-04 normal operation	Similar readings produce low conflict and a normal recommendation.
PRESS-04 faulty sensor	A single high outlier receives less influence and does not dominate the consensus.
PRESS-04 split conflict	The system selects the stronger group based on trust mass rather than only counting sensors.
PRESS-04 poor network	A stale high reading with weak communication does not blindly trigger shutdown.
PRESS-04 stale reading	Old evidence receives lower freshness and trust.
PRESS-04 emergency	Fresh, agreeing high readings support a shutdown recommendation.
Decision confidence bands	Medium confidence requests additional data, while low confidence defers the decision and holds the machine intent.
Configured sensor baseline	Initial readings remain in the learning or calibration state.
Configured sensor anomaly	A value far from the learned baseline is marked as unusual.
Configured sensor data quality	Old readings are marked stale, and unknown sensor IDs are rejected.
Sensor management	Removing a sensor also removes its associated readings.
Database migration	An older SQLite database gains the reporting-interval column.

Verification Results

At the time this report was prepared, the following verification results were recorded:

- Backend automated test suite: 15 tests passed.
- Frontend production build and TypeScript check: Passed.
- Frontend lint command: Completed with React initial-load effect warnings.
- Production build: Generated successfully, but Vite reported a bundle-size warning because the generated JavaScript chunk exceeded its default 500 kB warning threshold.

These results confirm the tested software behaviours. They do not establish statistical accuracy on real sensor data, production-scale real-time performance, or suitability for safety-critical use.

6. PROJECT DEMONSTRATION

6.1 Demonstration Environment

The application runs locally in a web browser using the FastAPI backend and Vite frontend.

The PRESS-04 simulator generates virtual sensor readings. Therefore, no physical IoT device is required for the demonstration.

The dashboard allows the user to select scenarios, view sensor readings, inspect recommendations, and review historical trends.

6.2 Explanation for a Project Presentation

The following explanation can be used during a project demonstration or presentation:

“This project demonstrates two related sensor-analysis workflows. In the PRESS-04 simulation, four virtual temperature sensors may report different values. The system checks each reading’s history, freshness, communication quality, agreement with other sensors, and operating context before calculating confidence.

Based on the available evidence, the system recommends whether to execute an action, request more readings, or defer the decision for review.

In the configurable sensor workflow, a user can define a sensor and its measurement unit, submit readings, and allow the software to learn that sensor’s normal range. The system can then identify readings that are unusual, stale, or received through a weak connection.

The prototype runs without physical hardware and does not control real equipment.”

7. RESULT AND DISCUSSION (COST ANALYSIS)

7.1 Results

The completed software prototype demonstrates the following:

- Conflicting simulated temperature readings can be grouped and evaluated using sensor quality and context rather than relying only on a fixed threshold or simple average.
- Decision confidence can lead to three different responses: execute, request additional data, or defer.
- User-defined sensor names and units can be stored without combining values from unrelated sensors.
- A separate baseline can be learned for each configured sensor.
- Later readings can be compared with the learned baseline to identify sufficiently unusual values.
- The configurable sensor workflow can distinguish an unusual reading from a stale reading or a weak network connection.
- The processing steps and recommendations can be inspected through the REST API and dashboard.

The current results are based on functional software tests. They are not measurements of real-world detection accuracy.

The prototype does not use a collected real-world sensor dataset. Therefore, the following metrics have not been calculated:

- Sensitivity
- Specificity
- False alarm rate
- Missed detection rate

These metrics would be required to evaluate the system's detection performance using labelled real-world data.

7.2 Cost Analysis

Cost of the Current Prototype

The current software demonstration does not require the purchase of dedicated IoT hardware.

It runs on an existing development computer and uses a local SQLite database. The repository contains the required Python and Node.js dependencies.

The following costs are not included in the current prototype:

- Cloud hosting
- Physical sensor devices
- Gateway hardware
- Commercial support
- Commercial licensing analysis

Cost Considerations for a Future Product

A commercial version would require a separate cost assessment.

The main cost areas would include:

- Hosting and managed database services.
- Storage, backups, data retention, monitoring, and network usage.
- Sensor and gateway hardware, installation, calibration, and replacement.
- Device onboarding, authentication, encryption, and certificate management.
- Customer accounts, access roles, and data separation.
- Engineering, testing, security review, support, and regulatory requirements.
- Integration and validation for each sensor model, measurement, and unit.

No total monetary cost is provided because the deployment assumptions and vendor prices have not been specified.

7.3 Discussion and Limitations

The project demonstrates an end-to-end software workflow for evaluating sensor evidence. However, it is not yet a production-ready, multi-customer product.

The main limitations are:

1. Temperature-specific trust engine: The multi-source trust and confidence calculation is designed specifically for the PRESS-04 temperature workflow.
2. Independent configured sensors: User-configured sensors use their own baseline detector. They do not participate in multi-sensor consensus and do not produce machine-level actions.
3. No unit conversion: Measurement names and units are stored as free-text labels. The system does not convert units or verify that the selected unit matches the intended physical measurement.
4. Initial baseline limitations: The first five readings are used to create a starting baseline. These readings are not guaranteed to represent normal operating conditions.
5. Prototype thresholds: The MAD threshold, freshness calculation, network-quality threshold, and machine temperature thresholds are initial prototype settings. They require calibration and evaluation for the intended application.
6. No customer separation: All users share the same local workspace. Authentication and tenant-level data separation are not implemented.

7. REST-only integration: The current system provides REST endpoints. MQTT and physical-device integrations are not included.
8. No real equipment control: The interface provides software recommendations only. It must not be connected to machinery as an automatic safety control without a separate engineering and certification process.

These limitations define the current scope of the project and help identify the next steps for development.

8. CONCLUSION

This project presents a software-based prototype that evaluates IoT sensor information before providing a decision recommendation.

The PRESS-04 temperature simulation demonstrates how the system can detect conflicts between multiple sensor readings, calculate context-aware trust, identify a dominant group of observations, estimate decision confidence, and select one of three responses: execute, request additional data, or defer the decision.

The separate configurable sensor workflow allows users to define their own sensors, specify measurement names and units, submit readings, and build a baseline for each sensor. It can then identify unusual, stale, or low-quality readings based on the sensor's recent history.

The prototype runs without physical hardware and includes a REST API, SQLite database storage, a multi-page React dashboard, and automated tests. These features make it suitable for academic demonstration and provide a foundation for further development.

The next stage of development should focus on testing the system with real, labelled sensor datasets, improving measurement-specific configuration and unit handling, supporting multi-sensor fusion for configured sensor groups, adding authentication and customer data separation, and integrating physical devices.

Further work is also required in security, deployment, and safety engineering. Until those improvements and validations are completed, the system should remain a non-safety-critical decision-support prototype.

9. Reference

S. No.	Author(s)	Title / Reference	Year	Relevance to Project
1	L. Atzori, A. Iera, and G. Morabito	"The Internet of Things: A Survey," Computer Networks, Vol. 54, No. 15, pp. 2787–2805.	2010	Provides background on IoT architecture, connected devices, and applications.
2	NIST	Networks of Things, NIST Special Publication 800-183.	2016	Explains IoT system components, sensing, computing, communication, and actuation.
3	F. Khalid et al.	Survey of trust and reputation systems in wireless sensor networks.	—	Supports the use of trust and reputation to evaluate sensor reliability.
4	P. J. Rousseeuw and C. Croux	"Alternatives to the Median Absolute Deviation," Journal of the American Statistical Association.	1993	Provides background on robust statistical methods for handling unusual sensor readings.

5	FastAPI	FastAPI Documentation.	—	Reference for REST API development and request validation.
6	React	React Documentation.	—	Reference for developing the dashboard interface using components.
7	SQLAlchemy	SQLAlchemy Documentation.	—	Reference for database models and object-relational mapping.
8	SQLite	SQLite Documentation.	—	Reference for the local database used to store sensor readings and decisions.